Project Proposal

“DRTHREED”

Design and Development of a Browser-Based 3D Environment Inspection Platform with Automated Scene Analysis, Configurable Collision Boundaries, and Terrain-Aware First-Person Navigation

Prepared by: Osanda Muthukumarana

Date: 11 October 2026

Abstract

This proposal presents the design and development of a browser-based 3D Environment Inspection Platform for importing, inspecting, configuring, and exploring 3D environments created in Blender. The platform will primarily accept self-contained GLB files exported from Blender, render their geometry and materials in a web browser, analyse the imported scene hierarchy, and generate an editable object inventory. A rule-based discovery process will assign probable categories—such as buildings, habitats, walls, ground, terrain, rocks, furniture, equipment, structural components, and decorative objects—using node names, hierarchy, bounding-box dimensions, proportions, surface orientation, metadata, and spatial relationships. Because this process is heuristic rather than guaranteed semantic understanding, users will be able to review and correct classifications.

The platform will provide visibility filtering, configurable collision boundaries, walkable-ground designation, spawn-point placement, and physics-based first-person navigation. It will distinguish visual meshes from approximate physics colliders and support no-collider, automatic, bounding-box, convex, static triangle-mesh, compound, and terrain-collider configurations. Users will be able to retain original collider dimensions, apply a per-side margin, or specify custom dimensions. The navigation system will support collision-aware walking and running, uneven-terrain traversal, slope handling, and subtle breathing, walking, and running camera motion. Compatible HDRI images will provide environment lighting. The initial release will focus exclusively on first-person observation; third-person camera modes and selectable animated characters are future extensions.

The proposed system will use client-side processing wherever possible, without requiring a mandatory backend or cloud account. Its evaluation will assess usability, reliability, rendering and navigation performance, collider correctness, and terrain traversal across representative indoor, industrial, habitat, and outdoor environments. The proposal distinguishes planned features from implemented functionality and does not claim results before evaluation is complete.

1. Introduction

1.1 Background

Three-dimensional environments are commonly authored in desktop modelling software such as Blender and exported for visualisation, simulation, training, architectural review, and interactive exploration. However, viewing a static 3D model is different from inspecting it as a navigable environment. Users often need to understand the structure of a scene, locate and isolate individual objects, distinguish floors and terrain from walls and equipment, and move through the environment without passing through solid geometry.

The glTF 2.0 format and its binary GLB container are widely used for efficient transmission of 3D scenes on the web. A GLB file can package the glTF JSON description and binary buffer data in one self-contained file, making it particularly suitable for browser-based import workflows. [web:1] Three.js provides GLTFLoader for loading glTF 2.0 assets, including both .glb and .gltf files. [web:8][web:9]

1.2 Proposed System

The proposed application is a browser-based 3D Environment Inspection Platform. Its home page at / will introduce the product and direct users to a separate inspection workspace at /inspector. In the workspace, users will import a GLB environment, inspect its scene hierarchy, discover and classify objects, control visibility, configure collision boundaries, designate walkable ground, place a spawn point, and explore the environment from a first-person perspective.

The platform is intended for architectural interiors, industrial facilities, laboratories, modular habitats, and outdoor landscapes with uneven terrain. It will emphasise local, client-side processing: a user should be able to import and inspect an environment without creating an account or uploading the asset to a cloud service, except where an optional feature explicitly requires network access.

1.3 Scope of the Initial Release

The initial version will focus exclusively on first-person observation and navigation. Third-person camera control and built-in animated character selection are future extensions, not initial-release requirements. The first release will primarily support self-contained GLB assets; support for .gltf files with external buffers, textures, and

other resources will depend on implementing a reliable resource-resolution workflow.

2. Problem Statement and Motivation

2.1 Problem Statement

Users who receive complex 3D environments exported from Blender often lack a convenient browser-based workspace that combines scene inspection, object discovery, collision configuration, and first-person exploration. Conventional 3D viewers can display geometry, but they may not provide an integrated workflow for understanding a scene hierarchy, correcting object categories, defining collision boundaries, marking walkable ground, and navigating an environment with terrain-aware movement.

This creates several practical difficulties. Users may need specialised desktop software to explore an asset interactively; they may struggle to interpret deeply nested scene graphs; and they may find it difficult to distinguish structural objects, terrain, equipment, and decorative meshes. Environments that were not authored for a game engine may also contain high-resolution visual geometry that is unsuitable for direct use as physics geometry.

2.2 Technical Motivation

A central issue is the difference between a visual mesh and a physics collider. A visual mesh represents the displayed surface of an object, whereas a collider is a simplified geometric representation used by a physics engine to detect contacts and prevent penetration. Rapier, for example, defines colliders as geometric shapes that generate contacts and collision events; solid colliders produce contact points and prevent interpenetration, while sensor colliders detect intersections without generating contacts. [web:2][web:4]

Using the full visual triangle mesh for every object can be computationally expensive and can produce unstable or unnecessarily complex collision behaviour. A bounding box is inexpensive but approximate; a convex hull is more shape-aware but cannot represent concavities; a static triangle-mesh collider can follow visible geometry more closely but is generally more costly. A practical inspection platform should

therefore let users choose an appropriate collider representation for each object or group.

2.3 Terrain and Navigation

Indoor floors and outdoor terrain require different navigation assumptions. An indoor floor may be approximated reliably by a flat or gently sloped ground collider, whereas an outdoor landscape can contain hills, rocks, ramps, steps, and uneven surfaces. First-person movement must detect the ground beneath the user, follow its surface, limit excessively steep slopes where appropriate, and prevent the camera from entering walls or other solid objects.

The proposed platform addresses these needs by combining object management, collider configuration, ground designation, spawn placement, and collision-aware first-person navigation in one browser-based workspace. It does not claim that existing tools cannot perform individual parts of this workflow; rather, it proposes an integrated, client-side workflow specifically oriented toward inspecting Blender-authored environments before or without transferring them into a game engine.

3. Aim and Objectives

3.1 Aim

To design and develop a modular, browser-based 3D Environment Inspection Platform that enables users to import Blender-authored GLB environments, analyse and organise their scene contents, configure collision boundaries and walkable ground, and inspect those environments through terrain-aware, physics-based first-person navigation.

3.2 Objectives and Verification

Objective	**Planned completion evidence / verification method**
Design a modular browser-based 3D inspection architecture.	Architecture diagram, module boundaries, data-flow documentation, and code review confirming separation of rendering, scene analysis, collider configuration, physics, UI, and persistence concerns.
Implement GLB import and scene	Load representative self-contained GLB

rendering.	files; verify geometry, materials, textures, transforms, and hierarchy are rendered without critical errors.
Build a scene-hierarchy and object-discovery interface.	Display the imported scene graph and object inventory; verify that selecting an inventory entry highlights or focuses the corresponding scene object.
Develop rule-based object categorisation.	Apply categorisation rules to test scenes; compare proposed categories with user-reviewed expected categories and record precision, recall, and correction rate.
Implement object visibility filtering.	Search, filter, select, hide, and show objects; verify that visibility changes immediately affect the rendered scene.
Provide configurable collider types and dimensions.	Create no-collider, bounding-box, convex, static-mesh, compound, and terrain colliders; verify collider creation, editing, and removal through visual debugging and physics tests.
Support collider margins and expanded collision footprints.	Apply a per-side margin to rectangular test colliders; verify that each horizontal dimension increases by twice the margin and that the physics footprint matches the configured value.
Allow custom collider dimensions.	Enter custom width, depth, and height values; verify that generated colliders use the entered dimensions within defined tolerance.
Support ground designation and spawn-point placement.	Mark suitable meshes or colliders as walkable ground and place a spawn point; verify that navigation begins at the spawn point and remains supported by designated ground.
Implement collision-aware first-person movement.	Walk and run through test environments; verify that the camera does not pass through enabled solid colliders and that movement responds to input.

Support uneven-terrain traversal and slope handling.	Traverse representative ramps, hills, and uneven terrain; record traversal success, ground alignment, and behaviour at defined slope thresholds.
Implement subtle breathing, walking, and running camera motion.	Enable and disable motion effects; verify that idle breathing, walking bob, and running motion are subtle, configurable, and do not impair usability.
Support HDRI environment lighting.	Load compatible HDRI images; verify that they affect scene lighting and reflections without replacing imported material textures.
Create a distinctive home page and separate inspection workspace.	Review / and /inspector routes; verify product introduction, workflow explanation, feature sections, and a functional link from home page to inspector.
Evaluate usability, reliability, and performance.	Conduct usability sessions, functional test cases, error logging, and performance measurements; report results only after testing is complete.

4. Functional Scope

4.1 Home Page and Navigation

The application will include a dedicated home page at / and an inspection workspace at /inspector. The home page will introduce the platform, explain its principal features, and provide a clear route into the inspection workspace.

- A distinctive 3D-inspired visual identity rather than a generic template.
- A concise explanation of the platform and its intended users.
- An overview of the import-to-inspection workflow.
- Key feature sections covering scene analysis, collision configuration, terrain navigation, and HDRI lighting.
- A prominent call to action linking to /inspector.
- Responsive layout and accessible navigation controls.

4.2 GLB/glTF Import

The initial release will primarily support self-contained GLB assets. The system will load supported geometry, materials, textures, transforms, and useful hierarchy information from the imported file. GLB is the binary container form of glTF and may package the JSON asset description and binary buffer in one file. [web:1]

The platform will also aim to support compatible .gltf files, but this depends on implementing a reliable resource-resolution workflow for external buffers, textures, and related files. Three.js documents GLTFLoader as an addon for the glTF 2.0 format and supports both .glb and .gltf assets. [web:8][web:9]

- File selection through a browser file picker or drag-and-drop interface.
- Loading progress indication and descriptive error messages.
- Support for scene-graph traversal and object naming.
- Optional support for Draco- or Meshopt-compressed assets where the required decoder workflow is implemented.
- Validation of unsupported, corrupted, or excessively large assets.

4.3 Object Discovery and Classification

After loading a scene, the platform will analyse its scene graph and create an object inventory. The inventory will expose object names, parent-child relationships, transforms, mesh information, bounding boxes, material summaries where available, and user-assigned categories.

Automatic categorisation will use a practical rule-based approach rather than an AI-based classifier. The system may consider object and node names, parent-child hierarchy, bounding-box dimensions, geometric proportions, surface orientation and extent, available metadata, and approximate spatial relationships.

- Buildings and habitats: large enclosed or multi-part structures with substantial vertical extent.
- Walls: elongated vertical surfaces with comparatively small thickness.
- Ground and terrain: broad horizontal or gently curved surfaces near the lower region of the scene.
- Rocks: irregular, compact, non-building forms with varied proportions.

- Furniture: objects with human-scale dimensions and recognisable naming or structural cues.
- Equipment: technical or functional objects identified through naming, metadata, hierarchy, or shape rules.
- Structural components: beams, columns, panels, supports, and similar construction elements.
- Decorative objects: small or visually distinctive objects that do not satisfy stronger structural rules.

The resulting categories are probabilistic or heuristic suggestions, not guaranteed semantic understanding. Users must be able to correct incorrect categories, assign custom labels, and retain full control over the scene. The proposal does not describe the system as AI-powered unless an actual AI-based classifier is included in the implementation plan.

4.4 Object Visibility and Filtering

Users will be able to search, filter, select, hide, and show scene objects. The interface will provide checkboxes for individual objects or suitable groups, and visibility changes will affect the actual rendered scene.

Visual visibility will be kept separate from collision state. Hiding an object will normally change only its rendered visibility; it will not automatically remove or disable its collider. A separate explicit control will allow a user to disable a collider when that behaviour is intended. This separation prevents accidental changes to the physics world when a user temporarily hides an object for inspection.

4.5 Configurable Collision Boundaries

Collision-boundary configuration is a major project feature. Users will be able to configure colliders for individual objects or groups, choosing the representation most appropriate for the object's role, shape, and performance requirements.

Collider mode	Intended use	Principal limitation
No collider	Visual-only objects, distant scenery, or objects that should not obstruct movement.	The object will not block or support navigation.
Automatic collider	A convenient default	The selected

selection	based on object properties.	approximation may require user review.
Simple bounding-box collider	Walls, floors, boxes, furniture, and compact objects.	Does not follow concave or irregular shapes.
Convex collider	Approximating reasonably convex objects with better shape awareness than a box.	Cannot represent concavities exactly.
Static triangle-mesh collider	Terrain, complex static architecture, and objects requiring closer geometric contact.	Higher computational cost and potential instability if used inappropriate.
Compound collider	Objects composed of several distinct parts.	Requires more configuration and can increase physics cost.
Ground or terrain collider	Walkable floors, landscapes, ramps, and other support surfaces.	Must be designated and validated as navigable ground.

The platform will distinguish clearly between the original visual mesh and the approximate physics collider. A collider is not assumed to be an exact copy of the visual geometry. Users will be able to retain original collider dimensions, expand a collider by a specified margin, or enter custom dimensions.

4.5.1 Footprint, Margin, and Expansion

For a rectangular footprint, width and depth are the two horizontal dimensions, while footprint area is their product:

$$\text{Area} = \text{width} \times \text{depth}$$

A per-side margin m expands each horizontal dimension by $2m$, because the margin is added to both opposing sides:

$$\text{Expanded width} = \text{width} + 2m$$
$$\text{Expanded depth} = \text{depth} + 2m$$

The expanded footprint area is therefore:

$$\text{Expanded area} = (\text{width} + 2\text{m}) \times (\text{depth} + 2\text{m})$$

The perimeter of a rectangular footprint is:

$$\text{Perimeter} = 2 \times (\text{width} + \text{depth})$$

For example, a habitat with a 4 m × 4 m footprint has an original area of 16 m² and a perimeter of 16 m. Applying a 0.25 m margin on every side produces a 4.5 m × 4.5 m collider, an expanded area of 20.25 m², and a perimeter of 18 m. The collider grows by 4.25 m², not simply by the area of the original footprint.

If a target area and one dimension are known, the other dimension can be calculated as:

$$\text{depth} = \text{target area} / \text{width}$$

If a target area and desired aspect ratio are known, the dimensions can be derived from the ratio, subject to practical rounding and scene-scale constraints. These calculations apply directly to rectangular bounding-box colliders; they do not imply that expanding a bounding box creates a mathematically exact offset of an arbitrary irregular or concave mesh.

4.6 Ground Designation and Spawn Placement

Users will designate one or more meshes or collider regions as walkable ground. The system will use this designation to determine valid support surfaces for first-person navigation. Ground designation will be separate from ordinary collider creation: an object may be visible and solid without being marked as walkable.

Users will place a spawn point in the inspection workspace. On entering navigation mode, the first-person camera will begin at the spawn point, subject to a validity check that prevents spawning inside a solid collider or above unsupported space.

4.7 First-Person Navigation

The navigation system will provide collision-aware first-person movement, including walking, running, turning, ground detection, and slope handling. Movement will use a physics representation of the environment rather than relying solely on visual geometry.

- Keyboard and pointer-based movement controls.
- Walking and running speeds with acceleration and deceleration.
- Collision detection against enabled solid colliders.
- Ground snapping and terrain following on walkable surfaces.
- Slope limits or reduced movement on excessively steep terrain.
- Optional step handling for small vertical discontinuities.
- Subtle idle breathing motion.
- Subtle walking and running camera bob.
- Configurable motion intensity and ability to disable camera motion.

4.8 HDRI Environment Lighting

The platform will support compatible HDRI environment images for image-based lighting. An HDRI can provide ambient illumination, reflections, and a more realistic visual context for imported materials. HDRI use will be optional and will not replace the imported GLB materials or textures.

- HDRI file selection and preview.
- Environment-lighting intensity control.
- Background visibility control.
- Rotation or orientation adjustment where supported.
- Fallback lighting when no HDRI is supplied.

5. Proposed System Architecture

5.1 Architectural Principles

The system will use a modular client-side architecture. The principal modules will be separated so that rendering, scene analysis, collider configuration, physics simulation, user-interface state, and local persistence can evolve independently.

- Rendering module: manages the WebGL scene, camera, lights, HDRI environment, and object visibility.
- Import module: reads GLB/glTF assets and converts them into an internal scene representation.
- Scene-analysis module: traverses the scene graph, computes bounds and object properties, and creates the object inventory.

- Classification module: applies transparent, editable rules to propose object categories.
- Collider module: creates, edits, visualises, and removes physics collider definitions.
- Navigation module: manages first-person input, character physics, ground detection, slope handling, and camera motion.
- UI state module: coordinates selection, filters, visibility, collider settings, and workspace layout.
- Local persistence module: optionally saves inspection settings in browser storage or an exportable project file.

5.2 Proposed Technology Stack

Layer	Proposed technology	Role
User interface	React or a comparable component-based framework	Home page, inspector workspace, panels, forms, and state-driven UI.
3D rendering	Three.js	WebGL rendering, scene graph management, lighting, HDRI support, and camera control.
Asset loading	Three.js GLTFLoader	Loading GLB/glTF assets and associated resources.
Physics	Rapier physics engine or a comparable WebGL-compatible physics library	Static colliders, terrain collision, character movement, contacts, and slope handling.
Programming language	TypeScript	Type safety, maintainability, and clearer module interfaces.
Build tooling	Vite or a comparable modern bundler	Development server, optimised production build, and module bundling.
Storage	Browser storage / file export	Optional local saving of inspection settings without a mandatory backend.

Three.js is appropriate because it provides established support for glTF/GLB loading and web-based 3D rendering. [web:8][web:9] Rapier is appropriate for collider-centred physics because its documentation explicitly distinguishes solid colliders, which generate contacts and prevent penetration, from sensor colliders, which detect intersections without generating contacts. [web:2][web:4]

6. Non-Functional Requirements

- Usability: Users should be able to complete import, object discovery, visibility filtering, collider configuration, ground designation, and navigation without specialised game-engine knowledge.
- Reliability: The application should handle invalid, incomplete, or unsupported files through clear error messages rather than unhandled failures.
- Performance: Rendering and navigation should remain responsive for representative environments; specific performance targets will be established during implementation and measured during evaluation.
- Privacy: The initial platform should process imported assets locally in the browser and should not require account creation or cloud upload.
- Portability: The application should operate in current major desktop browsers that support WebGL and required browser APIs.
- Maintainability: Modules should have clear responsibilities, typed interfaces, and testable boundaries.
- Accessibility: The interface should provide keyboard-accessible controls, readable contrast, and descriptive labels where practical.

7. Implementation Plan

Phase	Main activities	Planned output
Phase 1: Requirements and design	Finalise user workflows, architecture, data models, UI wireframes, and test plan.	Requirements specification, architecture diagram, wireframes, test plan.
Phase 2: Core application	Create home page, inspector route, layout, routing, and foundational UI components.	Navigable application shell.

Phase 3: Import and rendering	Implement GLB import, scene rendering, progress handling, error handling, and scene-graph inspection.	Working GLB viewer and object inventory.
Phase 4: Analysis and classification	Implement hierarchy analysis, bounding-box calculation, filtering, selection, visibility control, and rule-based categorisation.	Scene-analysis and object-management features.
Phase 5: Collision configuration	Implement collider modes, collider dimensions, margins, custom dimensions, visual debugging, and ground designation.	Configurable collision-boundary workflow.
Phase 6: Navigation	Implement spawn placement, first-person controls, collision-aware movement, terrain following, slope handling, and camera motion.	Interactive first-person inspection mode.
Phase 7: HDRI and refinement	Implement HDRI lighting, usability improvements, performance optimisation, and documentation.	Polished initial release candidate.
Phase 8: Evaluation	Execute usability sessions, functional tests, performance measurements, and defect resolution.	Evaluation report and final project documentation.

8. Evaluation Plan

8.1 Usability Evaluation

Usability will be evaluated through task-based sessions with representative users. Participants will be asked to import an environment, locate objects, adjust visibility, configure a collider, designate ground, place a spawn point, and navigate the scene. The evaluation will record task completion, time on task, errors, assistance required, and qualitative feedback.

8.2 Functional and Reliability Testing

Functional tests will verify each objective against predefined test cases. Reliability testing will include invalid files, unsupported formats, interrupted loading, large scenes, missing textures where applicable, and collider configurations that could create unsafe navigation conditions.

8.3 Performance Evaluation

Performance will be measured on representative GLB environments using browser profiling tools and application-level instrumentation. Candidate measures include initial load time, time to interactive inspection, rendered frame rate during navigation, collider-generation time, memory use where measurable, and navigation latency.

Performance targets will be defined before testing and reported as measured results. No performance claim will be presented as achieved until the evaluation has been completed.

8.4 Terrain Navigation Testing

Terrain tests will use controlled environments containing flat floors, ramps, hills, uneven ground, steps, and steep slopes. The tests will assess whether the user remains on designated ground, whether collision boundaries prevent penetration, and how the system behaves at defined slope thresholds.

9. Risks and Mitigation

Risk	Potential impact	Mitigation
Large or complex GLB files reduce browser	Slow loading, low frame rate, or unstable	Use loading indicators, progressive loading where

performance.	navigation.	feasible, collider simplification, level-of-detail strategies, and performance testing.
Rule-based classification produces incorrect categories.	User confusion or additional correction effort.	Make categories editable, explain that results are heuristic, provide search and filtering, and measure correction rates.
Visual meshes are unsuitable as physics geometry.	Poor performance or unstable collision behaviour.	Offer multiple collider types and encourage appropriate approximations.
Uneven terrain causes unnatural movement.	Reduced usability and immersion.	Implement ground detection, terrain following, slope limits, step handling, and configurable camera motion.
Browser compatibility limitations.	Reduced accessibility across devices.	Target current major desktop browsers and test WebGL-dependent functionality early.
Local persistence limits collaboration.	Users cannot easily share inspection settings.	Provide export/import of project settings as an optional future enhancement.

10. Future Extensions

- Third-person camera mode.
- Selectable animated characters and character-controller customisation.
- AI-assisted semantic object classification, only if an actual trained model or AI service is integrated.
- Collider optimisation suggestions and automatic convex decomposition.
- Multiplayer or shared inspection sessions.
- Backend-supported project storage and team collaboration.
- Measurement, annotation, sectioning, and report-generation tools.

- Extended support for external-resource glTF assets and compressed texture workflows.

11. Conclusion

The proposed browser-based 3D Environment Inspection Platform addresses the practical need to inspect, organise, configure, and navigate Blender-authored 3D environments without requiring specialised desktop software or a mandatory cloud backend. By integrating GLB import, scene-hierarchy analysis, rule-based object discovery, visibility control, configurable collision boundaries, ground designation, spawn placement, and terrain-aware first-person navigation, the system will provide a focused workflow for environment inspection.

The project will clearly separate proposed functionality from implemented functionality and will evaluate usability, reliability, performance, collider correctness, and terrain traversal before reporting outcomes. The modular client-side architecture is technically feasible using established web technologies, including Three.js for rendering and glTF/GLB loading and a physics engine such as Rapier for collider-based navigation.

12. References

1. Khronos Group. (2021). glTF 2.0 Specification. https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.pdf
2. Three.js. GLTFLoader documentation. https://threejs.org/docs/pages/GLTFLoader.html
3. Three.js. Loading 3D Models manual. https://threejs.org/manual/pages/loading-3d-models.html
4. Dimforge. Rapier documentation: Colliders. https://rapier.rs/docs/user_guides/templates/colliders/
5. Dimforge. Rapier JavaScript documentation: Colliders. https://rapier.rs/docs/user_guides/javascript/colliders/